The `lutorch_ex` library

Multi-head lookup tables in PyTorch, from one table to the full model

Anatoly Starostin, DeepSpike Inc.

October 2026

Abstract

`lutorch_ex` is a small PyTorch library that replaces the feed-forward network (FFN) of a transformer layer with lookup tables (LUTs). An FFN multiplies its input by two weight matrices. The replacement performs a simpler operation. It turns the input into a small number of sign bits, uses those bits as addresses into many small learnable tables, and adds up the rows that the addresses select. The lookup itself needs no matrix multiplication. In a sweep of eight lookup variants trained for 48,000 steps, with one seed per configuration, the best lookup variant of the library reaches a slightly lower validation loss than the dense FFN (1.0865 against 1.0894 bits per byte). Its 8-bit version reaches 1.0899 bits per byte, the same as the dense FFN within the variation between runs.

How the report is organized

The report builds the library from its smallest element to the full system, and each level contains the one before it (Figure 1). Sections 1 to 4 go through the levels: one table, the lookup variants (the rules for what a table returns), the heads and the multi-head LUT, and the wrapper, which gives the multi-head LUT the interface of an FFN. Two further sections apply to all of these levels and are not shown in the figure: Section 5 describes training, and Section 6 describes the same computation with 8-bit tables. Section 7 reports the experiments with the full model. Table 2 collects the notation.

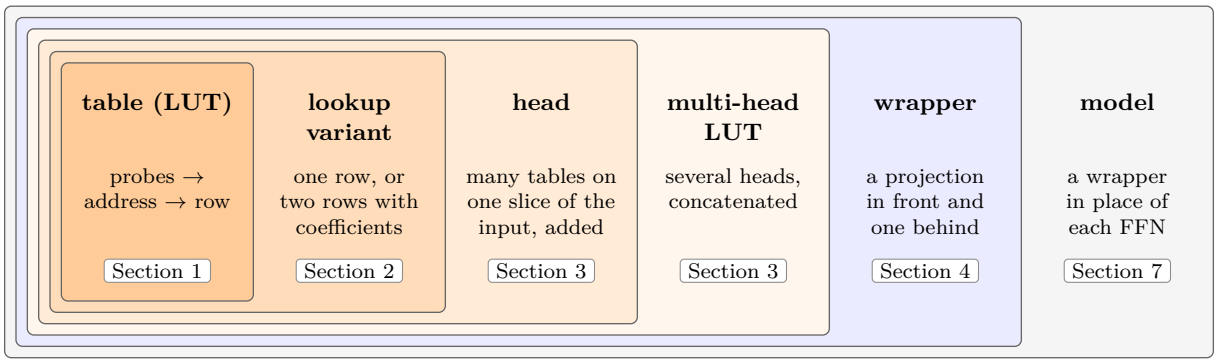


Figure 1: The levels of the library, from the smallest element to the full system. Each box contains the box to its left: a lookup variant is a table together with the rule for what it returns, a head is a group of tables, a multi-head LUT is a group of heads, and so on. Sections 1 to 4 follow the boxes from left to right, and Section 7 reports experiments with the full model.

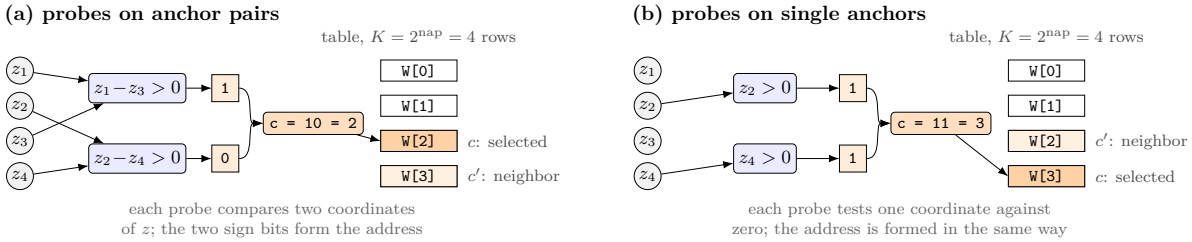


Figure 2: One lookup table with two probes (`nap`= 2), for each of the two kinds of probe. A probe yields one sign bit. The bits, most significant first, form the address c, which selects one learnable row of the table. The table has $K = 2^{\text{nap}} = 4$ rows. The probe with the smallest margin is the least certain one. If its bit is inverted, the address selects the neighbor row c' (shown in a lighter color). A weighted retrieval (Section 2) also fetches this row. (a) A probe on an anchor pair compares two coordinates of z. (b) A probe on a single anchor tests one coordinate against zero.

1 One table

This section describes the innermost box of Figure 1: a single lookup table.

A lookup table does its work with memory instead of arithmetic. A matrix layer multiplies its input by weights and adds up the products. A table instead computes an **address** from its input and returns the vector that is stored at that address. This operation is called a **lookup**. The address must be cheap to compute, and inputs that are similar to each other should usually receive the same address. A hash function with this property is called a locality-sensitive hash (LSH) [1]. The hash function used in this library consists of a small number of sign bits.

The construction comes from Izhikevich's *Spiking Manifesto* [2]. A spiking network stores a very large number of synapses, but at any moment it uses only the synapses of the neurons that have fired. A table lookup works in the same way: the table is large, and each input uses only one row of it. In the Manifesto, the information is carried by the order in which the neurons fire. An address made of sign bits encodes this order, because each bit records which of two neurons fired first.

From the input to an address. A table receives a vector $z \in \mathbb{R}^{d_{\text{in}}}$ as its input (Figure 2). It examines z with a small number of **probes**. The code calls the number of probes `nap`. Each probe computes one number u_j from z. There are two kinds of probe. A probe on an **anchor pair** (a_j, b_j) compares two coordinates of z, $u_j = z[a_j] - z[b_j]$. A probe on a **single anchor** a_j uses one coordinate, $u_j = z[a_j]$. An **anchor** is a position in z, that is, the index of one coordinate. The anchors are chosen at random when the table is created, and training does not change them.

Each probe produces one bit: the bit is 1 if $u_j > 0$, and 0 otherwise. The `nap` bits, written next to each other, form the address c. The address is an integer between 0 and $K - 1$, where $K = 2^{\text{nap}}$.

From the address to a row. The table has K rows, one for each address. Row c is a learnable vector $W[c] \in \mathbb{R}^{d_{\text{out}}}$. In the simplest case, the lookup returns this row, so the table maps the input z to the row $W[c(z)]$. A table with `nap` probes therefore divides its inputs into K classes, and it stores one vector for each class.

Margins and the neighbor row. The magnitudes $|u_j|$ are used as well. The magnitude $|u_j|$ is called the **margin** of bit j. It measures how far the input is from the point where bit j changes its value. The smallest margin of a table, $u^\star = \min_j |u_j|$, is called the **deciding margin**, and the bit with this margin is the least certain bit of the address. If this bit is inverted, the result is a second address c', which selects the **neighbor row**. The neighbor row is the row that the input

would select after the smallest change of the input that changes the address. A table passes three things on to the next level: the address c, the margins u_j, and the neighbor address c'. The lookup variants of Section 2 use no other information about the input.

Two kinds of probe. The kind of probe is chosen when a multi-head LUT (Section 3) is built, and all of its tables use the same kind. The default kind is the anchor pair. It is the comparison used in the Manifesto (its Eq. 1), in which the bit records which of two neurons fired first. The other kind is the single anchor, which tests the sign of one coordinate. Section 4 compares the two kinds, because the result of the comparison depends on what is placed in front of the tables.

2 Lookup variants: what a table returns

This section describes the next box of Figure 1. Section 1 ended with the simplest lookup, in which the table returns the selected row $W[c]$. A table can return more than this row. It can also fetch the neighbor row, and it can multiply each row by a coefficient. A **lookup variant** is the rule that fixes these choices. (The code calls a lookup variant a *cartridge*; this report uses that word only for the Python classes.) The lookup variants are interchangeable: when one lookup variant is replaced by another, nothing else in the system changes. Section 2.1 gives the single formula that all lookup variants share. The rest of the report needs only that part. Section 2.2 describes the lookup variants of the library one by one, and it can be skipped at first.

2.1 One formula, two ways to use it

For one table, every lookup variant computes a vector f, which is the **contribution** of the table to the output:

$$f = a\,W[c] + b\,W[c']. \tag{1}$$

Here $W[c]$ is the selected row, and $W[c']$ is the neighbor row, whose address differs from c in the least certain bit. The coefficients a and b are numbers. They depend on the input only through the margins of the table. The formula is used in two ways.

Hard lookup. The contribution is the selected row alone, that is, $a = 1$ and $b = 0$. The row is used as it is stored, without any multiplication. This is the cheapest possible lookup.

Weighted retrieval. Both rows are fetched, and each row is multiplied by its coefficient. The neighbor row receives a larger share of the sum when the deciding margin is smaller. The reason is that, with a small margin, a small change of the input would have selected the neighbor row instead.

A weighted retrieval uses Equation (1) both in training and in decode (inference). A hard lookup also uses the neighbor row and the coefficients, but only during training, where they are needed to compute the gradient (Section 5.1).

2.2 The lookup variants of the library

The library has two families of lookup variants (Table 1). The Manifesto lookup variants compute the coefficients from the deciding margin with an **uncertainty function** U. The argument

`uncertainty_func` chooses U: a fixed function (`"rational"`, the default), or a function with two learnable temperatures (`"softsign"`). The confidence lookup variants also use all the other margins of the table. The Manifesto lookup variants are available both as a hard lookup and as a weighted retrieval. The confidence lookup variants are available only as a weighted retrieval.

lookup variant	a, b	defined by	learned scalars
Manifesto, rational	$1-U$, U	$U = 0.5/(1+u^\star)$	none
Manifesto, soft-sign	$1-U$, U	$U = \sigma(-2\rho/T_{\text{sel}})$ with $\rho = u^\star/(T_{\text{soft}}+u^\star)$	T_{soft}, T_{sel}
Confidence, $n=1$	s, 0	$s = \big(\sum_j m_j\big)\exp\big(\gamma \sum_j \log\sigma(\beta m_j)\big)$	β, γ
Confidence, $n=2$	$s\,(1-v)$, $s\,v$	the same s, and $v = \sigma(-2u^\star/\tau)$	β, γ, τ

Table 1: The coefficients of the selected row (a) and of its neighbor (b) in Equation (1), for the lookup variants of the library. Here $u^\star$ is the deciding margin, $m_j = |u_j|$ are the margins of the table, σ is the logistic function, and n is the number of rows retrieved per table. In a hard lookup these coefficients are used only in training. Table 4 lists the classes.

Manifesto, rational uncertainty. The coefficient of the neighbor row is the fixed uncertainty function U of the Spiking Manifesto [2], applied to the deciding margin. U equals $\frac{1}{2}$ when the deciding margin is zero. In that case the two rows are equally plausible. U decreases toward zero as the margin grows. The hard lookup with this U, `ManifestoHardLUT`, is the rule of the Manifesto as it was published. The weighted retrieval, `ManifestoSoftLUT`, goes beyond the Manifesto. In the Manifesto, U is used only to compute gradients during training, and inference uses the plain sum of the selected rows (its Eq. 3 and Section III-A).

Manifesto, soft-sign uncertainty. This uncertainty function has the same role, but it is learnable. Two parameters, the temperatures T_{soft} and T_{sel}, replace the fixed constants of U. The first temperature rescales the deciding margin into a number ρ between 0 and 1. The second temperature turns ρ into the coefficient U, which is at most $\frac{1}{2}$. Both temperatures are learned. There is one pair of temperatures for each multi-head LUT (Section 3), and both start at the value 0.5. They can also be kept fixed. The same two classes, `ManifestoHardLUT` (hard lookup) and `ManifestoSoftLUT` (weighted retrieval), use this U when they are constructed with `uncertainty_func="softsign"`.

Confidence. The Manifesto lookup variants use only the deciding margin. The confidence lookup variants use all the margins of a table to estimate how much the table can be trusted for the present input. It multiplies the contribution of the table by a **confidence score** s, which is the score of LookupFFN [3]. The score is a product of two factors. The first factor is the sum of the margins. It grows as the input moves away from the points where the bits change their values. The second factor, the exponential, is equal to the product of $\sigma(\beta m_j)^\gamma$ over the probes of the table. Each term of this product is close to 1 when the margin is large, and it equals $2^{-\gamma}$ when the margin is zero. Every uncertain bit therefore reduces the contribution of the table. The scalar β determines which margins count as large, and the scalar γ determines how strong the reduction is.

The class `ConfidenceLUT` retrieves either one row or two rows per table ($n = 1$ or $n = 2$). With $n = 1$, the contribution of the table is the selected row multiplied by the score. With $n = 2$, the neighbor row is added as in the Manifesto weighted retrieval, and the score multiplies both rows. The coefficient v of the neighbor row is at most $\frac{1}{2}$. It depends only on the deciding margin and on

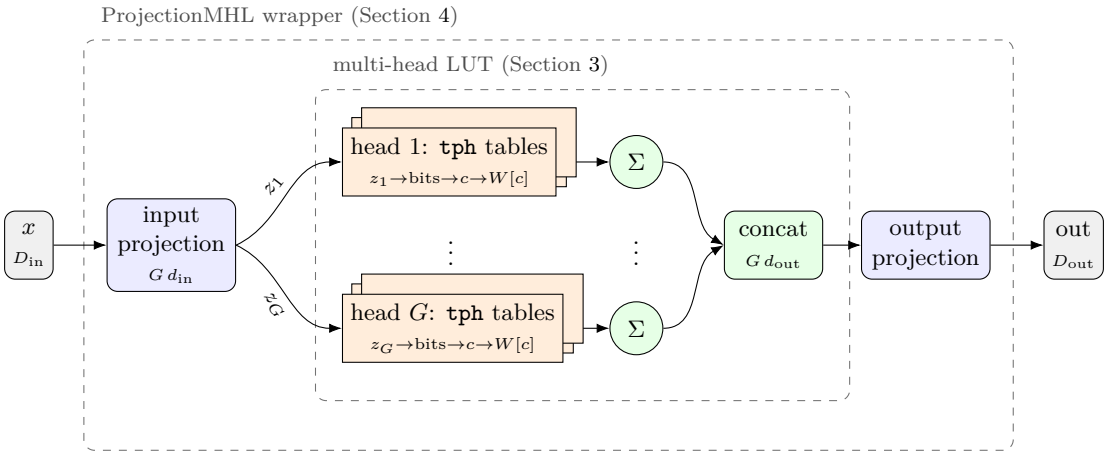


Figure 3: A multi-head LUT (inner dashed box, Section 3) inside the ProjectionMHL wrapper (outer dashed box, Section 4). The input projection ($D_{\text{in}} \to h_{\text{in}} d_{\text{in}}$) splits x into the input slices z_g of the heads. In each head the probes turn z_g into one address per table, and the contributions of the `tph` tables of that head are added (Σ). The G head outputs are concatenated, and the output projection ($h_{\text{out}} d_{\text{out}} \to D_{\text{out}}$) maps the concatenation to the output. (With $h_{\text{out}} = 1$, the head outputs are added into one slice instead.) The lookup variant determines how the contribution of a table is computed. The wrapper does not use the internals of the multi-head LUT.

a temperature τ. The scalars β, γ and τ are learned, and each multi-head LUT has one value of each. They are stored as logarithms, and their initial values are 2, 1 and 0.5.

3 Heads and the multi-head LUT

This section describes the next two boxes of Figure 1. One table divides its inputs into only K classes, so a model uses many tables and adds up their contributions. The tables are grouped into heads, and the heads are grouped into a multi-head LUT (the inner dashed box of Figure 3).

A head. A **head** is a group of tables that all examine the same input vector. The code calls the number of tables per head `tph`. Head g receives its own slice z_g of the input. The slice has d_{in} values, and all tables of the head apply their probes to it. Every table has its own anchors, so every table forms its own address. Table t of head g forms the address $c_{g,t}$ from z_g, and it returns its contribution $f_{g,t}$ according to Equation (1). The output of the head is the *sum* of these contributions:

$$y_g = \sum_{t=1}^{\text{tph}} f_{g,t} \ \in \mathbb{R}^{d_{\text{out}}}, \qquad g = 1, \ldots, G. \tag{2}$$

For a hard lookup, the output is the sum of the selected rows, $y_g = \sum_t W_{g,t}[c_{g,t}]$.

A multi-head LUT. A **multi-head LUT** consists of G heads that work side by side. The outputs of the heads are *concatenated* into one vector:

$$y = [\, y_1,\ y_2,\ \ldots,\ y_G \,] \ \in \mathbb{R}^{G\,d_{\text{out}}}. \tag{3}$$

As an alternative, the library can add the outputs of the heads into a single vector of width d_{out}, $y = \sum_g y_g$. Together with the projections of Section 4, the multi-head LUT replaces an FFN.

How large is a multi-head LUT? A multi-head LUT has $G \times$ `tph` tables, and each table has 2^{nap} rows. With 8 heads, 64 tables per head and `nap`= 8, which are the sizes used in Section 7, the total is $8 \times 64 \times 256 = 131{,}072$ rows. Every row holds d_{out} values. With $d_{\text{out}} = 48$, the tables therefore hold 6.3 million values. In the six-layer model of Section 7, the tables of the six layers hold 37.7 million values together, which is more than half of the 68.2 million parameters of the model. The tables are large, but one input vector uses only a small part of them: one or two rows from each table.

name	meaning	value in Section 7
One table (Section 1)		
z	input vector of a table; all tables of a head share it	
`nap`	number of probes per table, which is also the number of address bits	8
$K = 2^{\text{nap}}$	number of rows per table	256
u_j, $\lvert u_j \rvert$	value of probe j, and the margin of its bit	
$u^\star$	deciding margin: the smallest of the margins of a table	
c, c'	address of the selected row, and of its neighbor row	8 bits each
$W[c]$	row c of a table: d_{out} learnable values	
One lookup variant (Section 2)		
a, b	coefficients of the selected row and of its neighbor row	
f	contribution of one table, $a\,W[c] + b\,W[c']$	
n	number of rows retrieved per table by a confidence lookup variant (`read_top_n`)	1 or 2
Heads and the multi-head LUT (Section 3)		
`tph`	number of tables per head	64
G	number of heads in a multi-head LUT	8
d_{in}	width of the input slice z_g of a head	48
d_{out}	width of a row, which is also the width of the output y_g of a head	48
The wrapper (Section 4)		
D_{in}, D_{out}	input and output widths of the wrapper	384
h_{in}, h_{out}	number of input slices and of output slices; both equal G in the usual case	8
B	number of input vectors in a batch	

Table 2: Notation, in the order of the levels of Figure 1. Names in typewriter font are the names used in the code.

How these heads compare with attention heads. The name and the arrangement follow multi-head attention. In a Transformer, the vector of width d_{model} is projected into h attention heads. Each attention head works in its own narrow space of width d_{model}/h, the heads do not exchange information, and their outputs are concatenated and projected back to the model width. A multi-head LUT is arranged in the same way. It has G heads, each head works on its own narrow slice, the heads do not exchange information, and their outputs are concatenated as in Equation (3). The projections in front of the heads and behind them are the subject of Section 4. Even the sizes

are similar: the original Transformer has 8 attention heads of width 64 in a model of width 512, and the model of Section 7 has 8 heads of width 48 in a model of width 384.

The two kinds of head differ in what a head computes. An attention head combines information from different tokens: for each token, it forms a weighted sum of value vectors that come from the other positions of the sequence. A head of a multi-head LUT works on one token at a time, as an FFN does. Its output, Equation (2), is a sum of table rows selected by that token alone, and no other position is involved. The heads of this report are therefore not attention heads. Where both kinds appear, the report writes "attention heads" for the latter.

4 The ProjectionMHL wrapper: input and output projections

This section describes the wrapper, which is the last box of the library in Figure 1. A multi-head LUT works on G narrow slices, but the model around it works on a single vector of the model width. The class `ProjectionMHL`, the wrapper, connects the two (Figure 3). It places one linear map in front of the multi-head LUT and another linear map behind it. The first map is the **input projection**, and the second is the **output projection**. (The code calls them `compress` and `decompress`.) Seen from the outside, the wrapper takes a vector of width D_{in} and returns a vector of width D_{out}, as an FFN does. The two widths are separate arguments of the wrapper, and they may be different. The data path is

$$x \in [B, D_{\text{in}}] \xrightarrow{\substack{\text{input}\\ \text{projection}}} z \in [B, h_{\text{in}}, d_{\text{in}}] \xrightarrow{\substack{\text{multi-head}\\ \text{LUT}}} y \in [B, h_{\text{out}}, d_{\text{out}}] \xrightarrow{\substack{\text{output}\\ \text{projection}}} \text{out} \in [B, D_{\text{out}}].$$

Here the brackets give the shapes of the arrays, B is the number of input vectors in a batch, h_{in} and h_{out} are the numbers of input slices and output slices, and d_{in} and d_{out} are the widths of these slices. In the usual case, every head has its own input slice and its own output slice, so $h_{\text{in}} = h_{\text{out}} = G$.[1] The two projections have different purposes, which the next two subsections describe.

4.1 The input projection

The input projection decides what the probes test. A probe tests on which side of a hyperplane the input lies. The set of hyperplanes that the probes can test depends on the kind of probe, and on whether an input projection is present (Table 3).

Without an input projection, a probe on a single anchor can only test the sign of one coordinate of the raw input. Such a test splits the input space along one coordinate axis. A probe on an anchor pair compares two coordinates, so anchor pairs are much stronger in this case. With an input projection, every coordinate of z is itself a learned linear function of x. A probe on a single anchor then tests an arbitrary learned hyperplane. This is the "hyperplane LSH" of the Manifesto, with learnable anchor vectors (2, Section VIII-H). A probe on an anchor pair tests the difference of two such functions, which is also an arbitrary hyperplane. With an input projection, the two kinds of probe therefore test hyperplanes of the same type, and they differ only in the number of hyperplanes that are available. The d_{in} coordinates of a head give d_{in} different hyperplanes

[1] The two numbers can be different only when one of them is 1. The library rejects every other combination. With $h_{\text{in}} = 1$, the input projection produces a single slice of width d_{in}, and all G heads take this slice as their input. With $h_{\text{out}} = 1$, the outputs of the heads are added into a single slice of width d_{out}, and the output projection receives this slice.

	single anchors	anchor pairs
no input projection	$x[a] > 0$: a plane along a coordinate axis of the raw input	$x[a] - x[b] > 0$: coefficients ± 1 and zero bias, the form of the Manifesto
with input projection	$w_a^\top x + b_a > 0$: an arbitrary learned hyperplane	$(w_a - w_b)^\top x + (b_a - b_b) > 0$: also arbitrary, with up to $\binom{d_{\text{in}}}{2}$ distinct planes per head

Table 3: The hyperplane that one probe tests. Here w_a and b_a are row a of the input-projection matrix and the corresponding bias, so that $z[a] = w_a^\top x + b_a$.

to single anchors, but up to $\binom{d_{\text{in}}}{2}$ different hyperplanes to anchor pairs. When more hyperplanes are available, different tables are less likely to use the same few bits. Single anchors are therefore somewhat weaker than anchor pairs even with an input projection, but the difference is not large.

With a learned input projection, are anchor pairs still needed? In principle, they are not needed. A pair comparison $x[a] - x[b] > 0$ is a hyperplane test in which the coefficient is $+1$ at position a, -1 at position b, and zero at all other positions. A single anchor behind an input projection could test the same hyperplane if the corresponding row of the projection matrix had this form. In practice, the rows of the matrix do not take this form, because ordinary training has no reason to turn a dense row into a row with one $+1$, one -1, and zeros. To obtain the pair form from a learned projection, the library would need additional methods that it does not have. Examples are a strict regularizer, or a straight-through estimator that maps the matrix entries to ± 1. (A straight-through estimator is a training method that rounds a value in the forward pass, and computes the gradient in the backward pass as if the value had not been rounded.)

For this reason, the library keeps both kinds of probe. The anchor pair is the mechanism of the Manifesto in its original form: it compares the order of two coordinates. The anchor pair is also a special case that is cheap to compute, and the whole design is built to take advantage of it. One address bit costs two memory reads (one coordinate each) and one subtraction. The anchors are stored as integer positions, and the bit has no trainable parameters of its own. The kernels of the library (the low-level routines that carry out the computation) work in this way. A general hyperplane for every bit would instead cost a dot product of length d_{in}, and it would need d_{in} parameters. In the example of Section 3, with $d_{\text{in}} = 48$, anchor pairs need about 4,000 subtractions per token and layer, whereas general hyperplanes would need about 200,000 multiply-adds. The single anchor was added to the library later, for flexibility. It is even cheaper than the anchor pair, but it is also somewhat weaker.

4.2 The output projection

The output projection keeps the rows narrow. Section 3 showed that the number of rows is determined by the numbers of heads, tables and probes. The size of the tables then depends on the row width d_{out}. If the rows had the full width of the model, $d_{\text{out}} = 384$, the tables of that example would hold 50 million values per layer. With $d_{\text{out}} = 48$ they hold 6.3 million values. The rows are therefore kept narrow, and the output projection afterwards maps the concatenated outputs of the heads to the model width. The cost of the output projection is one small dense matrix per layer, with 0.15 million parameters.

Switching a projection off. When the widths already match, one of the two projections can be

replaced by the identity map. The library does not allow both projections to be replaced at the same time. By default, both projections are present. The output projection is initialized to zero, and the input projection is initialized with random values from $\mathcal{N}(0, 0.02)$.

4.3 In code

The classes of the library correspond to the levels of Figure 1 (Table 4). Each lookup variant is a class. An object of this class is a complete multi-head LUT that uses one rule for the contribution of a table. All lookup variants have the same interface, which is defined by the shapes of the arrays: a lookup variant maps an array of shape $[B, h_{\text{in}}, d_{\text{in}}]$ to an array of shape $[B, h_{\text{out}}, d_{\text{out}}]$. The class `MultiHeadLUT` defines this interface. The wrapper relies only on these shapes, and it does not use the internals of a lookup variant.

All lookup variants are also derived from one base class, `ManifestoLUT`. The base class holds the tables, in an array W of shape $G \times \texttt{tph} \times K \times d_{\text{out}}$, and the anchors. It also provides the things that every lookup variant needs: the selected row, the margins, the neighbor row, and the routing of the result of each head to its position in the output. The class of a lookup variant then defines two things of its own. The first is the choice of the coefficients in Equation (1). The second is the function that training differentiates (Section 5.1). Each of these classes, the **reference** classes, is written in plain PyTorch and runs on the CPU and on CUDA. Each also exists in a version whose name begins with `Fused`. A fused class computes the same mathematics in hand-written CUDA kernels. It needs a CUDA input, and it never falls back to PyTorch: an input that its kernels do not cover raises an error.

The wrapper accepts any lookup variant, and the example below uses `ConfidenceLUT`. The wrapper maps an array of shape $[N, D_{\text{in}}]$ to an array of shape $[N, D_{\text{out}}]$, so a batch of sequences must first be flattened into a list of vectors:

```
import torch
from spiky.lutorch_ex import LUTSpec, ConfidenceLUT, ProjectionMHL

spec = LUTSpec(h_in=8, h_out=8, tph=64, nap=8, d_in=48, d_out=48) # 8 heads of 64 tables
mhl = ConfidenceLUT(spec, seed=1, read_top_n=2) # the cartridge; 2 rows per table
ffn = ProjectionMHL(mhl, d_model=384) # or input_dim=..., output_dim=... (may differ)
x = torch.randn(4, 128, 384) # (B, T, D) activations
y = ffn(x.reshape(-1, 384)).reshape(x.shape) # torch.Size([4, 128, 384])
```

Any other lookup variant class can be used in place of `ConfidenceLUT`; a fused class needs the input on a CUDA device.

5 Training: gradients and regularizers

Sections 1 to 4 describe the forward computation, that is, how the output is computed from the input. This section describes what training needs in addition: a way to compute the gradient through a lookup, and two regularizers. Both topics concern the inner boxes of Figure 1, that is, the tables and their lookup variants.

level	class	role
lookup variant, Manifesto	`ManifestoHardLUT`, `ManifestoSoftLUT`	hard lookup, weighted retrieval; `uncertainty_func="rational"` (the default) or `"softsign"`
lookup variant, confidence	`ConfidenceLUT`, `QuantisedConfidenceLUT`	weighted retrieval with $n = 1$ or $n = 2$ rows (`read_top_n`); the second trained for int8 tables (Section 6)
lookup variant, fused	the prefix `Fused` before any of these four class names, for example `FusedManifestoHardLUT`	the same mathematics in hand-written CUDA kernels; CUDA input only
lookup variant, int8 decode	`Deployed` `QuantisedConfidenceLUT`	inference only, built from an export of `QuantisedConfidenceLUT` (Section 6)
multi-head LUT	`MultiHeadLUT`	the interface of every lookup variant: $[B, h_{\text{in}}, d_{\text{in}}] \to [B, h_{\text{out}}, d_{\text{out}}]$
	`ManifestoLUT`	the base class of every lookup variant: tables, anchors, addresses, margins, routing of the head outputs
	`LUTSpec`	the sizes of a multi-head LUT: the numbers of heads, tables per head and probes per table, and the widths
wrapper	`ProjectionMHL`	input projection, multi-head LUT, output projection

Table 4: The classes of the library, by level.

5.1 How the gradient reaches the input

Training needs the derivative of the contribution of a table, Equation (1), with respect to the input z of the table. The addresses c and c' are integers, and an integer-valued quantity has no useful derivative. The coefficients a and b, however, are smooth functions of z, because they depend on z through the margins. The gradient can therefore pass through the coefficients.

Weighted retrieval. In a weighted retrieval, the gradient with respect to the input passes through a and b only. The selected row and the neighbor row both contribute to the output, so both rows are updated by training.

Hard lookup. In a hard lookup, the coefficients are constants ($a = 1$, $b = 0$), so the output has no useful derivative with respect to the input. A hard lookup therefore uses a different function in the backward pass. In the backward pass, and only there, it uses Equation (1) with the coefficients of the weighted retrieval with the same uncertainty function. A function that is used in this way is called a **surrogate**: it is a smooth function that replaces a function without a useful derivative. The surrogate provides the gradient with respect to the input. The update of the table itself goes to the selected row c only.

Manifesto. With the coefficients $a = 1 - U$ and $b = U$ of Table 1, differentiating Equation (1)

gives

$$\frac{\partial \tilde{f}}{\partial z[a_{j^\star}]} = -\frac{\partial \tilde{f}}{\partial z[b_{j^\star}]} = U'(u^\star)\ \mathrm{sgn}(u_{j^\star})\ (W[c'] - W[c]).$$

Here $j^\star$ is the deciding probe, $a_{j^\star}$ and $b_{j^\star}$ are its two anchors, and the tilde indicates the surrogate. The gradient with respect to the input is therefore nonzero only at the two anchors of the deciding probe, and it has opposite signs at the two anchors (Eqs. 4 and 5 of the Manifesto). This holds for both uncertainty functions, because both depend on the input only through the deciding margin. With `"softsign"`, the two temperatures receive a gradient as well.

Confidence. In a confidence lookup variant, the gradient reaches the input only through the score s, which depends on all `nap` margins, and for $n = 2$ also through v, which depends on the deciding margin. In this way, a table learns how much confidence to assign to its output. The gradient never tells the table that a different row would have been a better choice.

5.2 Regularizers

Two regularizers are implemented in the base class `ManifestoLUT`, so every lookup variant with a trainable table can use them (all except `DeployedQuantisedConfidenceLUT`, which is inference only). Both are switched off by default. The first regularizer acts on the rows of a table, and the second acts on whole tables.

Row total variation. The $K = 2^{\mathrm{nap}}$ rows of a table can be pictured as the corners of a cube with `nap` dimensions, with one dimension for each address bit. Two rows are neighbors when their addresses differ in a single bit, as the addresses c and c' do. When the input changes so that one bit changes its value, the input moves from one row to a neighbor of that row. This is the smallest change of the input that changes the address. It is therefore reasonable to require that such a change alters the output only slightly. The regularizer expresses this requirement as a penalty on the mean squared difference between neighboring rows:

$$\mathcal{R}_{\mathrm{TV}} = \frac{1}{T \cdot \mathrm{nap} \cdot 2^{\mathrm{nap}-1}} \sum_{t=1}^{T} \sum_{(c,c')} \left\| W_t[c] - W_t[c'] \right\|_2^2.$$

Here $T = G \cdot$ `tph` is the number of tables. The inner sum runs over all pairs of rows whose addresses differ in a single bit, and each pair is counted once. A table has $\mathrm{nap} \cdot 2^{\mathrm{nap}-1}$ such pairs, which explains the normalization factor. The method `ManifestoLUT.cell_tv` computes this penalty for one multi-head LUT. The function `cell_tv_penalty(model)` averages the penalty over the multi-head LUTs of the model, and the training loss adds λ times this average. The penalty acts only on the values in the tables, and it does not change the forward pass.

Table dropout. When the dropout rate p is greater than zero (the argument `table_dropout_rate`), training removes whole tables at random. For every sample, every head and every table, an independent random decision keeps the table with probability $1 - p$. The outputs of the tables that are kept are multiplied by $1/(1 - p)$, so the expected output does not change. A table that is removed contributes nothing to the output or to the gradient for that sample. In a confidence lookup variant, the dropout mask multiplies the score s, which has the same effect. In decode, and whenever gradients are switched off, no table is removed and no output is rescaled.

6 The int8 lookup and decode

Sections 1 to 4 describe the computation in floating point. This section shows how the same computation is carried out with 8-bit tables and integer arithmetic, and how a trained model is stored for decode. The change affects every level of Figure 1 from the table to the wrapper: the rows of a table, the coefficients of the lookup variant, the sum of a head, and the output projection.

Why quantize? Most of the parameters of a LUT model are in its tables. In the example of Section 3, the tables hold 6.3 million values per layer. In the whole model, they hold 37.7 million of the 68.2 million parameters. When the tables are stored as 8-bit integers (int8) instead of 32-bit floating-point numbers (fp32), they need only a quarter of the memory. `QuantisedConfidenceLUT` is the confidence lookup variant, with $n = 1$ or $n = 2$, that is trained so that it can be decoded in this form; `FusedQuantisedConfidenceLUT` computes the same in CUDA kernels. Its quantization mode is named `p2_int8`, which stands for powers of two and int8. The mode is selected by the argument `quant_mode="p2_int8"`.

What `p2_int8` does. The mode converts the weighted retrieval $s(1-v)\,W[c] + s\,v\,W[c']$ into integer arithmetic. It does this in three steps, one for each of the three inner levels of Figure 1.

1. **The rows of a table become int8.** Each row is stored as

$$\hat{W} = \text{clamp}\,\big(\,\text{round}(W/2^e),\,-128,\,127\big), \qquad e = \lceil \log_2 \max |W| \rceil - 7.$$

   The integer e is called the **shared exponent**. There is one shared exponent for each head and each output component, and the maximum is taken over all tables and all rows of the head. This choice of e makes the largest entry as large as an 8-bit integer allows.

2. **The coefficients of the lookup variant become powers of two.** The retrieval can be written as $s(1-v)\,(W[c] + r\,W[c'])$ with the ratio $r = v/(1-v) = e^{-2u^\star/\tau}$. The ratio is rounded to a power of two, 2^{-q}, with $q = \text{round}\,\big(2u^\star/(\tau \ln 2)\big)$. The common factor $s(1-v) = s/(1+r)$ is rounded to $2^{k'}$, with $k' = \text{round}(\log_2 s - c_q)$ and $c_q = \log_2(1 + 2^{-q})$. A table then adds $2^{k'}\hat{W}[c] + 2^{k'-q}\hat{W}[c']$. This operation needs two shifts and one addition, and it needs no multiplication. With $n = 1$ there is no neighbor row: the score itself is rounded, $k' = \text{round}(\log_2 s)$, and a table adds $2^{k'}\hat{W}[c]$.

3. **The sum of a head becomes an integer sum.** The shifts are counted in units of 2^{-6}, so that every shift is a shift to the left. The sum over the tables is kept in a 32-bit integer accumulator. Only one floating-point operation remains: the shared exponent is applied once, after the sum over the tables, as the factor 2^{e-6} for each output component.

The scheme has a few limits. The exponents q and k' are computed from the margins in floating point. Only the table lookup and the sum use integer arithmetic. The exponent q is limited to at most 64, and c_q is set to 0 when $q \geq 8$. The exponent k' is kept between -3 and 4. If k' is below this range, the table is skipped for that input, but it still receives its gradient. If k' is above this range, it is set to 4. The neighbor row is omitted when $q > 3$, that is, when its coefficient is less than $\frac{1}{8}$ of the coefficient of the selected row.

Training is quantization-aware. The model is trained with the rounding already applied: the forward pass in training uses the rounded tables and the rounded coefficients. Rounding has no useful derivative, so both roundings use a straight-through estimator. Like the surrogate of Section 5.1, a straight-through estimator uses one function in the forward pass and another function

in the backward pass. The forward pass computes with the rounded values, and the backward pass computes the gradient as if the values had not been rounded. The gradient therefore goes to the floating-point **master tables**, from which the int8 tables are derived, and to the exact coefficients $s(1-v)$ and $s\,v$. For this reason, the scalars β, γ and τ continue to be trained.

Decode. Because the rounding is already applied during training, decode computes the same numbers as training, except for round-off errors. Preparing a trained model for decode therefore changes only the way in which the model is stored. The function `export_deployment` writes the int8 tables to a file and discards the floating-point master tables. It also multiplies a copy of the output-projection matrix by the factor 2^{e-6}, so that the shared exponent becomes part of that matrix. This multiplication is exact, because multiplying a floating-point number by a power of two changes only its exponent. As a result, no separate step has to apply the shared exponent at run time. The file contains the int8 tables, the anchors, the scalars β and γ (and τ when $n = 2$) and the two projections; the rest of the model is stored as it is. In the example, the tables of one layer shrink from 24 MiB to 6 MiB (a MiB is 2^{20} bytes). The function `load_deployment` first calls a function supplied by the user, `make_skeleton`, which builds the model on PyTorch's `meta` device, a placeholder device on which arrays have shapes but occupy no memory. It then fills the model from the file, so the floating-point tables are never allocated in memory. Each quantized multi-head LUT becomes a `DeployedQuantisedConfidenceLUT`. In decode, it returns the integer accumulator, and the output projection applies the shared exponent that was included in it. On a CUDA input, the integer lookup is one hand-written kernel; on a CPU input, it is the PyTorch definition of the same integer arithmetic.

```
export_deployment(model, "m.lxq")
dep = load_deployment("m.lxq", make_skeleton, device="cuda").eval()
```

7 Experiments: eight lookup variants and the dense FFN

This section describes the outermost box of Figure 1, the full model. It answers two questions. How well do the lookup variants learn, compared with each other and with the dense FFN that they replace? And how fast do they run?

7.1 Setup

The test model is a six-layer GPT. In every layer, the FFN is replaced by a `ProjectionMHL` wrapper that contains the lookup variant under test. Eight configurations were trained, and they differ only in the lookup variant. Each of them uses the fused class of its lookup variant. A ninth run, the baseline, keeps the dense FFN. Table 5 lists the settings. The configuration files, the training scripts, and a summary of every run (including the training time and the peak memory) are available on the branch at https://github.com/anatoli-starostin/spiky/tree/research/lutorch_ex_fused_ablations/experiments/lutorch_ex_fused.

The metric is `val_bpb`, the validation loss in bits per byte (bpb). It is the cross-entropy on a fixed held-out set, measured in bits and divided by the number of *bytes* of the text that the tokens represent. (This is the bits-per-byte measure of nanochat.) Because the loss is divided by the number of bytes and not by the number of tokens, the metric can be compared between models with different tokenizers. A lower value is better.

item	value
model	six-layer GPT (pre-norm blocks, rotary position embeddings, untied output layer), $d_{\text{model}} = 384$, 6 attention heads; 68.2M parameters with the multi-head LUTs, 35.8M with the dense FFN
dense FFN	$384 \to 1536 \to 384$ with GELU, no bias, no dropout
wrapper	square: $D_{\text{in}} = D_{\text{out}} = d_{\text{model}} = 384$
multi-head LUT	$h_{\text{in}} = h_{\text{out}} = 8$ heads, `tph`$= 64$, `nap`$= 8$, $d_{\text{in}} = d_{\text{out}} = 48$, anchor pairs
initialization	seed 1; tables uniform in $[-10^{-3}, 10^{-3}]$ per head, input projection $\mathcal{N}(0, 0.02)$, output projection zero
regularizers	multi-head LUTs only: row total variation with $\lambda = 10$, and table dropout with $p = 0.2$ (Section 5.2)
optimizer	AdamW, learning rate 3×10^{-4}, $\beta = (0.9, 0.95)$, weight decay 0.1 (none on the tables and on the parameters with fewer than two dimensions), gradient clipping at 1.0
schedule	48,000 steps; cosine schedule, 10% warm-up, floor at $0.1\times$ the peak rate
batch	sequences of 512 tokens; 24,576 tokens per step (LUT runs: device batch 12, gradient accumulation 4; dense run: device batch 48), 1.18 billion tokens in all
precision	fp32 throughout
validation	every 500 steps, on a fixed held-out set of 4,788 sequences
data	the *NanoChat* setup: the ClimbMix text corpus as prepared by nanochat's data script, 170 shards; the runs read less than one pass of it
tokenizer	nanochat's RustBPE tokenizer, with a vocabulary of 32,768 entries trained by nanochat on the same corpus; the text is streamed and tokenized on the fly into sequences of 512 tokens, aligned to the beginning-of-sequence (BOS) token
library	`lutorch_ex` of 12–14 October 2026, in which `uncertainty_func` was still called `weight`; the mathematics is the one described here

Table 5: The settings of the nine runs.

7.2 Results

Table 6 and Figure 4 show five results. Each configuration was run once. `FusedConfidenceLUT` with $n = 1$ was also run with seeds 2 and 3, which gave 1.09682 and 1.09460 bpb. The standard deviation of the three runs, 0.0013 bpb, shows how much a result can vary from one run to the next.

1. **A weighted retrieval is better than a hard lookup.** With both uncertainty functions, the weighted retrieval has the lower loss: by 0.014 bpb with the rational one, and by 0.028 bpb with the soft-sign one. The learnable soft-sign function helps the weighted retrieval (by 0.009 bpb), but not the hard lookup, which it makes worse by 0.005 bpb.
2. **Two rows are better than one.** Among the confidence lookup variants, the form that also retrieves the neighbor row ($n = 2$) is better than the form with a single scored row ($n = 1$), by 0.010 bpb. Both forms multiply their rows by the score. This result therefore compares one row with two rows, and it does not compare a hard lookup with a weighted retrieval. The

FFN replacement	lookup	params	`val_bpb`	Δ vs. dense
dense FFN, $384 \to 1536 \to 384$, GELU	none	35.8M	1.08936	
`FusedConfidenceLUT` $n = 2$	scored weighted retrieval	68.2M	**1.08653**	−0.0028
`FusedQuantisedConfidenceLUT` $n = 2$	the same, in int8	68.2M	1.08986	+0.0005
`FusedConfidenceLUT` $n = 1$	scored single row	68.2M	1.09698	+0.0076
`FusedQuantisedConfidenceLUT` $n = 1$	the same, in int8	68.2M	1.09928	+0.0099
`FusedManifestoSoftLUT`, soft-sign	weighted retrieval with U	68.2M	1.10954	+0.0202
`FusedManifestoSoftLUT`, rational	weighted retrieval with U	68.2M	1.11863	+0.0293
`FusedManifestoHardLUT`, rational	hard lookup	68.2M	1.13242	+0.0431
`FusedManifestoHardLUT`, soft-sign	hard lookup	68.2M	1.13780	+0.0484

Table 6: The nine runs at 48,000 steps, one seed each. Every run improves until its last evaluation, so `val_bpb` is both the final and the best value. "Soft-sign" and "rational" are the values of `uncertainty_func`. The baseline has the same backbone (6 layers, $d_{\text{model}} = 384$, 6 attention heads), data, tokenizer, schedule, batches of 24,576 tokens and seed, with a dense $384 \to 1536 \to 384$ GELU network as the FFN and no regularizer (run `00_vanilla`). The int8 lookup variants are trained with int8 rows and power-of-two coefficients (Section 6). Each fused class has the mathematics of its reference class.

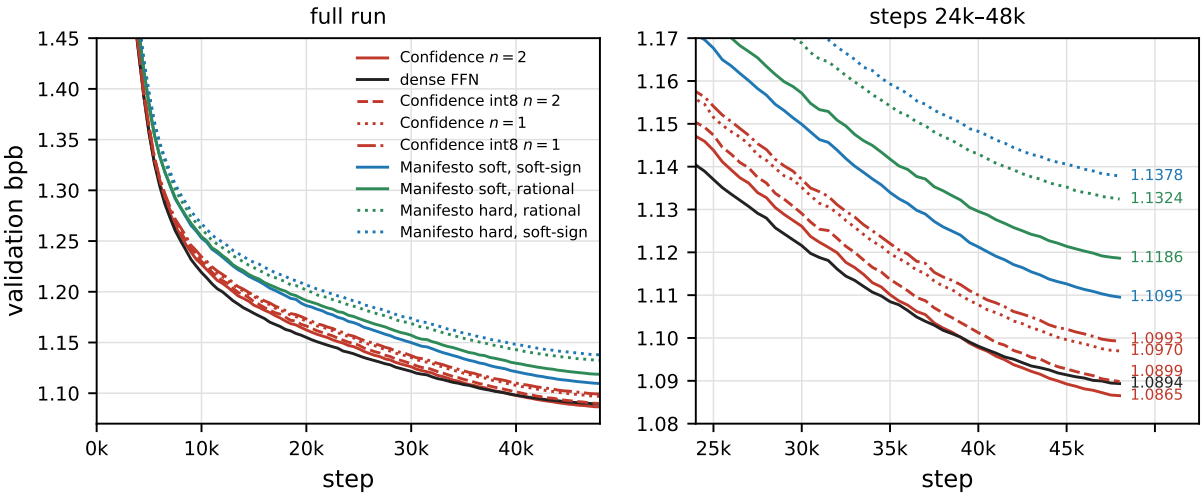


Figure 4: Validation loss in bits per byte for the nine runs. Left: the full run. Right: the second half, with the final value of each run. Color marks the family (red confidence, blue Manifesto with the soft-sign uncertainty function, green Manifesto with the rational one, black the dense FFN). Solid lines retrieve two rows per table, and dotted lines one row (the two hard lookups, and confidence with $n = 1$). The dashed and the dash-dotted lines are the int8 lookup variants with $n = 2$ and $n = 1$.

same result holds after quantization, by 0.009 bpb. Both differences are seven to eight times the variation from run to run.

3. **The confidence score improves the result.** The Manifesto lookup variants weight the two rows according to the uncertainty of one bit only. The confidence lookup variants add the confidence score, and they do better: `FusedConfidenceLUT` with $n = 2$ is ahead of the best

Manifesto run (the weighted retrieval with the soft-sign function) by 0.023 bpb, and even the single scored row ($n = 1$) is ahead of it by 0.013 bpb. The order of the eight lookup variants does not change after step 6,500.

4. **Quantization costs little.** With int8 rows and power-of-two coefficients, the loss increases by 0.0033 bpb at $n = 2$ and by 0.0023 bpb at $n = 1$, and the tables need only a quarter of the memory.
5. **Only the best lookup variant is better than the dense FFN.** `FusedConfidenceLUT` with $n = 2$ has a lower loss than the dense FFN, by 0.003 bpb, which is about twice the variation from run to run. Its curve crosses below the curve of the dense FFN only after step 38,500. Its int8 counterpart ends 0.0005 bpb above the dense FFN, which is within the variation from run to run. The other lookup variants are worse by 0.008 to 0.048 bpb. The LUT models have 1.9 times as many parameters as the dense model. For each token, however, a multi-head LUT uses only one or two rows of 48 values from each table.

7.3 Benchmarks: RTX 5090 and H100

Table 7 gives the timings of a single multi-head LUT, measured without the model around it, for the classes that the sweep trained: the eight fused classes, and `DeployedQuantisedConfidenceLUT`, the int8 form that is used for decode. Two reference classes are added for comparison. The numbers are taken from the benchmark records in the repository, `src/spiky/lutorch_ex/benchmarks/results_2026-10-15/` (`rtx5090.md` and `h100.md`). They were measured on an RTX 5090 and on an H100 with the same code and the same settings, using the benchmark module of the library, `spiky.lutorch_ex.benchmarks.bench_snapshot`. The records use 16 heads, twice as many as the model of the sweep, and 8,192 input vectors.

The reference classes are written in plain PyTorch, to be easy to understand, and the fused classes are tested against them. The Manifesto reference classes are not in the table: their training step runs without compilation, so it is not comparable with the others. The fused classes run only on CUDA, and the records contain no CPU timings.

The table shows the following.

- **Most of a training step is the backward pass.** A fused training step takes 1.5 to 2.5 ms on the RTX 5090 and 1.9 to 3.1 ms on the H100. According to the records, the backward pass takes 74 to 82% of it on the RTX 5090.
- **A second row costs time in decode.** On the RTX 5090, the decode time grows by a factor of about 1.7 from one row to two rows (from 0.28 to 0.49 ms for Manifesto, from 0.29 to 0.51 ms for confidence, and from 0.34 to 0.57 ms for its int8 counterpart in training form).
- **The int8 decode is the fastest decode of the confidence lookup variants.** `DeployedQuantisedConfidenceLUT` reads the int8 tables with one kernel. On the RTX 5090 it takes 0.24 ms with one row and 0.34 ms with two rows, against 0.29 and 0.51 ms for the floating-point `FusedConfidenceLUT`; the H100 shows the same order. Its tables also need a quarter of the memory (6 MiB instead of 24 MiB per layer of the sweep).
- **The soft-sign uncertainty function costs training time.** It makes the training step of the Manifesto classes longer by 11 to 25% on the RTX 5090, and by 29 to 45% on the H100. On the RTX 5090 it changes the decode time by 6% or less; on the H100 it makes the decode of the hard lookup 38% longer.

	RTX 5090		H100		
lookup variant	decode ms	step ms	decode ms	step ms	peak MiB
`FusedManifestoHardLUT`, rational	0.281	1.545	0.447	2.072	96.0
`FusedManifestoHardLUT`, soft-sign	0.297	1.929	0.619	3.012	96.0
`FusedManifestoSoftLUT`, rational	0.492	2.086	0.650	2.380	96.0
`FusedManifestoSoftLUT`, soft-sign	0.509	2.321	0.706	3.066	96.0
`FusedConfidenceLUT` $n = 1$	0.293	1.517	0.402	1.987	96.4
`FusedConfidenceLUT` $n = 2$	0.507	2.414	0.722	3.103	96.4
`FusedQuantisedConfidenceLUT` $n = 1$	0.335	1.474	0.688	1.915	128.4
`FusedQuantisedConfidenceLUT` $n = 2$	0.565	2.497	0.952	2.744	128.4
`DeployedQuantisedConfidenceLUT` $n = 1$	0.242	–	0.353	–	–
`DeployedQuantisedConfidenceLUT` $n = 2$	0.336	–	0.476	–	–
reference classes, for comparison					
`ConfidenceLUT` $n = 2$	1.801	5.389	1.751	6.697	840.0
`QuantisedConfidenceLUT` $n = 2$	2.585	12.489	4.484	19.123	1142.4

Table 7: Timings of one multi-head LUT alone, with $h_{\text{in}} = h_{\text{out}} = 16$, `tph`$= 64$, `nap`$= 8$, $d_{\text{in}} = d_{\text{out}} = 48$, in fp32, on 8,192 random input vectors, from the benchmark records `benchmarks/results_2026-10-15/` of the library (torch 2.9.1, CUDA 13.0). *Decode* is a forward pass in evaluation mode without gradients. *Step* is one training step, a forward and a backward pass. Each time is the median wall-clock time of 100 steps, after 20 warm-up steps and with the L2 cache flushed before each step; the median is taken over 3 rounds and averaged over the passes of the record (3 on the RTX 5090, 2 on the H100). *Peak* is the peak memory of a training step above what is allocated before it; it is the same on both GPUs. The deployed class has no training step.

- **The fused classes are faster and need less memory.** At $n = 2$, the training step of `FusedConfidenceLUT` is 2.2 times faster than that of the compiled reference class `ConfidenceLUT` on both GPUs, and it needs 8.7 times less memory. For the int8 lookup variant the training step is 5 to 7 times faster.
- **The RTX 5090 is faster than the H100 here.** On every line of the table except the decode of `ConfidenceLUT`, the RTX 5090 takes less time, by a factor of up to about 2.

7.4 Which lookup variant to use

For the best accuracy, use the confidence lookup variant with $n = 2$ (`ConfidenceLUT`, or `FusedConfidenceLUT` on a GPU). In the sweep it is the only lookup variant with a lower loss than the dense FFN, by a small margin. For decode with int8 tables, train `QuantisedConfidenceLUT` with $n = 2$ (`FusedQuantisedConfidenceLUT` on a GPU), export it, and decode with `DeployedQuantisedConfidenceLUT`; in the sweep it matches the dense FFN within the variation from run to run. The Manifesto lookup variants are useful mainly as points of reference. A hard lookup reads one row per table without a multiplication, but it is less accurate than the weighted retrieval with the same uncertainty function (Table 6), and in the benchmarks the int8 decode with one row is faster still.

References

[1] P. Indyk and R. Motwani. “Approximate nearest neighbors: towards removing the curse of dimensionality.” Proceedings of the 30th ACM Symposium on Theory of Computing (STOC), 1998, pp. 604–613.

[2] E. M. Izhikevich. “Spiking Manifesto.” arXiv:2512.11843, v2, December 2025.

[3] Z. Zeng, M. Davies, P. Pulijala, K. Sankaralingam, V. Singh. “LookupFFN: Making Transformers Compute-lite for CPU Inference.” ICML 2024.